

Selección y Justificación de Estilos y Patrones Arquitectónicos

El diseño arquitectónico del sistema se fundamenta en la necesidad de procesar grandes volúmenes de datos (hasta 16GB) de manera eficiente, garantizando atributos de calidad como **performance (tiempo de respuesta), escalabilidad y correctitud**. A partir de estos drivers, se seleccionaron y aplicaron diversos estilos y patrones arquitectónicos de manera complementaria, los cuales se describen y justifican a continuación.

Estilos Arquitectónicos

1. Arquitectura distribuida

Se adopta una arquitectura distribuida debido al alto volumen de datos a procesar, el cual no puede ser manejado eficientemente por un único nodo. Este estilo permite dividir la carga de trabajo entre múltiples máquinas de la sala 104M (workers), logrando un paralelismo real y reduciendo significativamente el tiempo de ejecución.

Relación con atributos de calidad:

- Mejora el performance al permitir procesamiento concurrente.
- Habilita escalabilidad horizontal al agregar nuevos nodos.

Ventajas:

- Procesamiento paralelo eficiente.
- Escalabilidad mediante adición de recursos.

Limitaciones:

- Mayor complejidad en comunicación y coordinación.
- Manejo más complejo de fallos.

Trade-off:

Se sacrifica simplicidad del sistema monolítico a cambio de mayor capacidad de procesamiento y escalabilidad.

2. Cliente-Servidor

Se utiliza el estilo cliente-servidor para separar la interacción del usuario del procesamiento del sistema. El cliente realiza solicitudes de consulta, mientras que el backend distribuido procesa y responde.

Relación con atributos de calidad:

- Mejora mantenibilidad y organización del sistema.
- Permite evolución independiente del frontend.

Ventajas:

- Separación clara de responsabilidades.
- Facilita el desarrollo y mantenimiento.

Limitaciones:

- Dependencia de un punto de entrada central.

Trade-off:

Se introduce una capa adicional para mejorar modularidad y claridad del sistema.

3. Arquitectura orientada a eventos (Event-Driven)

Se adopta un enfoque orientado a eventos para desacoplar la capa de visualización del procesamiento. Los distintos componentes publican eventos como progreso o resultados parciales, los cuales pueden ser consumidos por el módulo de visualización.

Relación con atributos de calidad:

- Mejora extensibilidad.
- Reduce acoplamiento entre componentes.

Ventajas:

- Flexibilidad para agregar nuevos consumidores de eventos.
- Bajo acoplamiento.

Limitaciones:

- Complejidad en trazabilidad y depuración.

Trade-off:

Se incrementa la complejidad del sistema a cambio de mayor flexibilidad y desacoplamiento.

4. Pipeline de procesamiento distribuido

El sistema sigue un flujo estructurado en etapas: cliente -> broker -> master -> workers -> master -> resultado. Este pipeline permite organizar el procesamiento en fases bien definidas.

Relación con atributos de calidad:

- Mejora performance al permitir optimización por etapas.
- Facilita paralelización.

Ventajas:

- Claridad en el flujo de datos.
- Modularidad en el procesamiento.

Limitaciones:

- Latencia acumulada entre etapas.

Trade-off:

Se introduce sobrecarga de coordinación a cambio de mayor organización y capacidad de optimización.

5. Arquitectura por componentes

El sistema se organiza en módulos independientes (client, broker, master, worker, visualization, partitioner, slice), cada uno con una responsabilidad específica.

Relación con atributos de calidad:

- Mejora mantenibilidad y extensibilidad.

Ventajas:

- Alta cohesión.
- Bajo acoplamiento.

Limitaciones:

- Mayor esfuerzo inicial de diseño.

Trade-off:

Se invierte más en diseño para facilitar evolución futura.

Patrones Arquitectónicos

1. Broker

El patrón Broker se utiliza para desacoplar al cliente del sistema distribuido. El broker actúa como intermediario en la comunicación, gestionando solicitudes y respuestas.

Relación con atributos de calidad:

- Mejora escalabilidad y flexibilidad.

Ventajas:

- Desacoplamiento entre componentes.
- Facilita integración de nuevos nodos.

Limitaciones:

- Puede convertirse en cuello de botella.

Trade-off:

Centraliza la comunicación para simplificar interacción, a costa de introducir un punto crítico.

2. Master-Worker

El patrón Master-Worker se emplea para distribuir el procesamiento de datos. El master coordina la ejecución, asigna tareas y consolida resultados, mientras los workers procesan en paralelo.

Relación con atributos de calidad:

- Mejora performance mediante paralelismo.
- Permite escalabilidad horizontal.

Ventajas:

- Distribución eficiente de carga.
- Paralelismo controlado.

Limitaciones:

- Dependencia del nodo master.

Trade-off:

Centralización del control para simplificar coordinación.

3. Data Partitioning

El dataset se divide en particiones independientes que pueden ser procesadas por diferentes workers. Esta división se realiza mediante offsets o segmentos del archivo.

Relación con atributos de calidad:

- Fundamental para escalabilidad.
- Reduce uso de memoria por nodo.

Ventajas:

- Procesamiento paralelo.
- Manejo eficiente de grandes volúmenes de datos.

Limitaciones:

- Complejidad en manejo de límites de partición.

Trade-off:

Mayor complejidad en partición a cambio de mejoras significativas en rendimiento.

4. Map-Reduce (simplificado)

Se implementa un modelo Map-Reduce donde los workers realizan cálculos parciales (map) y el master consolida los resultados finales (reduce).

Relación con atributos de calidad:

- Optimiza procesamiento de datos masivos.

Ventajas:

- Modelo claro y probado para procesamiento distribuido.

Limitaciones:

- Posible cuello de botella en la fase de reducción.

Trade-off:

Simplicidad conceptual frente a posibles limitaciones en agregación final.

5. Publisher–Subscriber

Este patrón se utiliza para notificar eventos a la capa de visualización sin acoplarla al procesamiento interno.

Relación con atributos de calidad:

- Mejora extensibilidad.

Ventajas:

- Bajo acoplamiento.
- Flexibilidad en consumo de eventos.

Limitaciones:

- Mayor complejidad en gestión de eventos.

Trade-off:

Se incrementa la complejidad a cambio de mayor desacoplamiento.

6. Proxy remoto

Se utiliza un mecanismo de proxy remoto para abstraer la comunicación entre nodos distribuidos, permitiendo invocar operaciones remotas como si fueran locales.

Relación con atributos de calidad:

- Simplifica el desarrollo distribuido.

Ventajas:

- Transparencia en la comunicación.
- Facilita implementación.

Limitaciones:

- Overhead de comunicación remota.

Trade-off:

Se sacrifica eficiencia en comunicación a cambio de simplicidad de desarrollo.

Conclusión

La combinación de estilos y patrones arquitectónicos seleccionados permite responder de manera efectiva a los principales drivers del sistema: procesamiento eficiente de grandes volúmenes de datos y escalabilidad horizontal. La arquitectura prioriza la distribución del cómputo, el desacoplamiento entre componentes y la modularidad, aceptando como trade-off una mayor complejidad en la coordinación y comunicación entre nodos.